3 A MEMORY MODEL FOR LLVM

This section informally describes a modified IR-level memory model that: enables high-level optimizations while still supporting low-level code; does not restrict movement of pointer arithmetic instructions (which remain pure functions); and, does not inhibit any standard integer optimizations. Section 4 formalizes the new model.

3.1 Deferred Bounds Checking

A drawback of LLVM's current inbounds pointer checking is that it prevents reordering of pointer arithmetic instructions and allocation functions:

```
char *p = malloc(4);   // (val=0x10, obj=p)
char *q = malloc(4);   // (val=0x14, obj=q)

char *r = (char*)((int)p + 5);  // (val=0x15, obj=*)
char *s = r +inb 1;             // (val=0x16, obj=q)
*s = 0; // OK
```

In this example, `s` is a valid pointer (i.e., it is in bounds of an object). However, if we move the definitions of `r` and `s` across that of `q`, `s` becomes out-of-bounds, and thus gets assigned **poison**.[2]

Constraining the movement of instructions is not desirable, since it inhibits optimizations like code hoisting. LLVM, as it turns out, freely moves pointer arithmetic instructions around. This is unsound. Our new model fixes the problem by instead using *deferred* bounds checking, in contrast with LLVM's current *immediate* bounds checking.

In deferred bounds checking, we allow out-of-bounds pointers to be created and manipulated; undefined behavior is only triggered when such a pointer is dereferenced. It is now OK to reorder pointer arithmetic across allocation functions in the previous example:

```
char *p = malloc(4);            // (val=0x10, obj=p)

char *r = (char*)((int)p + 5);  // (val=0x15, obj=*)
char *s = r +inb 1;             // (val=0x16, obj=*, inb={0x15,0x16})

char *q = malloc(4);            // (val=0x14, obj=q)

*s = 0; // OK since 0x15 and 0x16 are inbounds of same object
```

For pointers with `obj=*`, we now track a set of addresses that have to be within bounds of the same object when the pointer is dereferenced. On every inbounds pointer arithmetic operation, we record in the `inb` field the base pointer as well as the resulting pointer. A memory access operation is UB if not all the addresses in `inb` are within bounds of the same object. Therefore, the inbounds check is *delayed* until the pointer is dereferenced. While deferred bounds checking achieves the same effect as immediate bounds checks, it allows free movement of pointer arithmetic instructions since they now do not depend on the memory state.

Precision could be further increased by replacing `*` provenance with the object(s) the cast pointer refers to. In the first example above we could define `r` to have value `(val=0x15, obj=q)` instead. However, this is also a form of immediate bounds checking with the same movement restriction. Moreover, some addresses are in bounds of two objects, like `0x14` in the example (corresponds to `p + 4` and `q`), adding further complexity to the model. Therefore, we do not use these semantics.

[2]Note that while this example is not correct in memory models with data-flow provenance tracking, it is ok in our model. Even though we build a pointer into `q` based on `p`, this might be the result of the compiler propagating an equality that established that `p == q + 4`.